

SQL – simple e-commerce system

In this project, I created a sample relational database for a simple e-commerce system. It was designed as part of my software tester portfolio – to demonstrate my ability to work with databases, write SQL queries, and create data-driven test scenarios.

PROJECT GOAL

This project demonstrates:

- working knowledge of SQL and relational databases,
- ability to work with table relationships,
- skills in creating test data,
- writing queries to verify and analyze data.

TECHNOLOGIES USED

- **phpMyAdmin** – for database management,
- **SQL** – standard queries.

DATABASE STRUCTURE

Tables:

users – customers of the store,

products – available products,

orders – orders placed by users,

order_items – details of products within orders.

Relationships:

users 1:N orders

orders 1:N order_items

products 1:N order_items

Test cases

Test Case 1 – Uniqueness of email addresses

Objective: To verify that there are no duplicate email addresses in the *users* table.

Steps:

1. Execute the SQL query:

```
SELECT email, COUNT(*) AS count
FROM Users
GROUP BY email
HAVING count > 1;
```

Expected result: The query returns no results.

Notes: Duplicate email addresses can cause issues with login and communication.

Actual result: The query returns no results. The test passed.

✅ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0064 sekund(y).)

```
SELECT email, COUNT(*) AS count FROM Users GROUP BY email HAVING count > 1;
```

Test Case 2 – Users with no orders

Objective: Check if there are users who registered but have not placed any orders.

Steps:

1. Execute the SQL query:

```
SELECT * FROM Users  
WHERE user_id NOT IN (SELECT user_id FROM Orders);
```

Expected result: List of users without orders or an empty list.

Notes: Depending on business requirements, the absence of orders may be acceptable or require intervention.

Actual result: The list contains one record. One user is registered but has not placed any orders.

✓ Pokazano wiersze 0 - 0 (1 total, Wykonanie zapytania trwało 0,0004 sekund(y).)

```
SELECT * FROM Users WHERE user_id NOT IN (SELECT user_id FROM Orders);
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Liczba wierszy: 25 ▾ Filter rows:

Extra options

	user_id	name	email	registration_date
☐	11	Karolina Koralewska	karolinak@example.com	2023-01-06

Można również edytować większość wartości klikając dwukrotnie bezpośrednio na nich.

☐ Check all ☐ Z zaznaczonymi. [Edit](#) [Copy](#) [Delete](#) [Export](#)

Test Case 3 – Orders with no products

Objective: Check if every order contains at least one product.

Steps:

1. Execute the SQL query:

```
SELECT * FROM Orders  
WHERE order_id NOT IN (SELECT DISTINCT order_id FROM Order_Items);
```

Expected result: No results.

Notes: An order without products is a logical error.

Actual result: No results found. The test passed.

✅ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0010 sekund(y).)

```
SELECT * FROM Orders WHERE order_id NOT IN (SELECT DISTINCT order_id FROM Order_Items);
```

Test Case 4 – Products with zero stock that have been ordered

Objective: Check if users order products whose current stock level is zero.

Steps:

1. Execute the SQL query:

```
SELECT order_items.order_id, products.name, products.stock
FROM order_items
INNER JOIN products
ON order_items.product_id = products.product_id
WHERE products.stock = 0;
```

Expected result: No results found.

Notes: It may indicate issues in inventory management if products with zero stock have been ordered.

Actual result: An order was placed containing a product with zero stock. The order was saved in the database despite the product being unavailable.

Test failed – the system should not allow ordering products that are out of stock.

✓ Pokazano wiersze 0 - 0 (1 total, Wykonanie zapytania trwało 0,0009 sekund(y).)

```
SELECT order_items.order_id, products.name, products.stock FROM order_items INNER JOIN products ON order_items.product_id = products.product_id WHERE products.stock = 0;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Liczba wierszy: 25 | Filter rows:

Extra options

order_id	name	stock
10	Klawiatura mechaniczna	0

Test Case 5 – Consistency of the order total

Objective: Verify that the sum of (*product price* × *quantity*) matches the *total* value in the *orders* table.

Steps:

1. Execute the SQL query:

```
SELECT orders.order_id, orders.total AS declared_total,  
SUM(products.price * order_Items.quantity) AS calculated_total  
FROM orders  
JOIN order_Items ON orders.order_id = order_items.order_id  
JOIN products ON order_Items.product_id = products.product_id  
GROUP BY orders.order_id  
HAVING declared_total != calculated_total;
```

Expected result: All *total* values in the *orders* table should be equal to the sum of (*price* × *quantity*) for all products in each order.
The query should return no rows.

Actual result: Five records were returned, indicating that five orders have incorrectly calculated totals – the *total* value in the *orders* table does not match the sum of order items (*price* × *quantity*).

This error indicates data inconsistency – the system is incorrectly saving or updating the total field.

Test failed – the data is inconsistent and requires correction.

✓ Pokazano wiersze 0 - 4 (5 total, Wykonanie zapytania trwało 0,0015 sekund(y).)

```
SELECT orders.order_id, orders.total AS declared_total, SUM(products.price * order_Items.quantity) AS calculated_total FROM orders JOIN order_Items ON orders.order_id = order_items.order_id JOIN products ON order_Items.product_id = products.product_id GROUP BY orders.order_id HAVING declared_total != calculated_total;
```

☐ Profilowanie [\[Edytuj w linii \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Liczba wierszy: 25 | Filter rows:

Extra options

order_id	declared_total	calculated_total
3	149.89	139.89
4	4897.99	3027.99
6	138.99	139.89
7	2698.00	2499.00
10	199.00	448.00

Queries used for analyzing data

- What was the average order value?

SELECT AVG(total) FROM orders;

Pokazano wiersze 0 - 0 (1 total, Wykonanie zapytania trwało 0,0004 sekund(y).)

```
SELECT AVG(total) FROM orders;
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Liczba wierszy: 25 ▾ Filter rows:

Extra options

AVG(total)

1301.215000

- Which products have the highest number of orders?

```
SELECT products.name, SUM(order_items.quantity) AS total_ordered_quantity
FROM order_items
INNER JOIN products
ON order_items.product_id = products.product_id
GROUP BY products.name
ORDER BY total_ordered_quantity DESC;
```

Pokazano wiersze 0 - 9 (10 total, Wykonanie zapytania trwało 0,0112 sekund(y).)

```
SELECT products.name, SUM(order_items.quantity) AS total_ordered_quantity FROM order_items INNER JOIN products ON order_items.product_id = products.product_id GROUP BY products.name ORDER BY `total_ordered_quantity` DESC
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Liczba wierszy: 25 ▾ Filter rows:

Extra options

name	total_ordered_quantity ▾ 1
Mysz Logitech	4
Pendrive 64GB	2
Smartfon Samsung	2
Powerbank 10000mAh	2
Laptop Lenovo	1
Monitor LG 24"	1
Sluchawki Sony	1
Kamera internetowa	1
Router WiFi TP-Link	1
Klawiatura mechaniczna	1

- Products with stock quantity less than or equal to 3.

```
SELECT name FROM products  
WHERE stock <= 3;
```

 Pokazano wiersze 0 - 1 (2 total, Wykonanie zapytania trwało 0,0005 sekund(y).)

```
1 SELECT name FROM products  
2 WHERE stock <= 3;
```

☒ Enable foreign key checks

Go

Cancel

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all

 | Liczba wierszy:

25 ▼

 Filter rows:

Przeszukaj tę tabelę

Extra options

					name
☐	 Edit	 Copy	 Delete		Klawiatura mechaniczna
☐	 Edit	 Copy	 Delete		Kamera internetowa